

基于股票数据的均值——方差模型优化及算法对比：

拉格朗日乘数法 VS 蒙特卡洛模拟法

1 数据来源及检验

本节将详细阐述研究中使用的数据来源、时间范围以及进行数据检验的具体步骤。

1.1 数据来源

研究基于当前市场的发展趋势和未来前景，选取 A 股市场中五支具有代表性的上市公司股票作为研究对象，通过多行业的股票组合，分析不同板块间的相关性与风险分散效果，能够更全面地展现均值-方差模型的应用价值。选取的股票是四个不同的行业中表现较好的五支，其中，新能源行业（比亚迪、宁德时代）正处于快速增长阶段，代表科技成长性强、风险收益较高的资产类型；电子零部件制造（捷荣技术）代表科技硬件供应链的中小市值股票，波动性较高，可在组合中体现行业轮动；医疗器械行业（迈瑞医疗）则具备持续稳定的盈利能力，被视为经济周期中较为稳健的成长股；能源公用事业（中国广核）以稳定现金流著称，对经济波动敏感度低，可在市场不确定性中提供稳健收益，是典型的低风险配置资产。五只股票覆盖了增长型、周期型与防御型资产，通过这样的分散组合能够有效体现跨行业分散投资的优势，也更有助于均值-方差模型展示资产间风险分散的具体效果。

本研究所有历史股票价格数据均来源 wind，确保数据的准确性与可靠性。研究所使用的数据为股票日度收盘价，为了捕捉市场近期波动特征，数据采集时间区间为 2024 年 4 月 5 日至 2025 年 11 月 5 日，共计约 18 个月。该时间范围足以计算出稳定可靠的年化收益率和波动率。

1.2 数据检验

为满足均值-方差模型的输入要求，需要对原始收盘价数据进行检验。为检验资产之间的相关性结构，计算并绘制了收益率相关性矩阵。通过相关性分析能够观察不同行业股票之间的联动性，为理解组合的风险分散来源提供依据。

2 模型建立

本章基于均值-方差模型，为投资组合优化问题搭建数学框架。模型核心在于将风险

与收益的权衡，转化为一个在权重配置约束下，以方差相关函数为目标的优化问题。

2.1 收益与风险的量化

为建立优化模型，首先需对资产的收益与风险进行量化。

采用日对数收益率作为基础度量，因其具备连续复利与时间可加性，更符合金融理论：

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

其中 P_t 为第 t 日收盘价。协方差矩阵 Σ 由 σ_{ij} 元素构成，第 i 项资产的预期收益率 μ_i ，方差 σ_i^2 以及资产 i 与 j 间的协方差 σ_{ij} ，计算如下：

$$\mu_i = \frac{1}{T} \sum_{t=1}^T r_i^t \quad \sigma_i^2 = \frac{1}{T-1} \sum_{t=1}^T (r_i^t - \mu_i)^2 \quad \sigma_{ij} = \frac{1}{T-1} \sum_{t=1}^T (r_i^t - \mu_i)(r_j^t - \mu_j)$$

2.2 投资组合的建模

定义各资产的资金配置比例权重向量 $w = (\omega_1, \omega_2, \dots, \omega_N)^T$ ，并满足完全投资约束：

$$\sum_{i=1}^N \omega_i = 1, \text{ 即 } w^T \mathbf{1} = 1$$

在此约束下，投资组合的预期收益率 μ_p 与组合风险 σ_p^2 分别为：

$$\mu_p = w^T \mu \quad \sigma_p^2 = w^T \Sigma w$$

2.3 优化目标的确立

根据不同的投资目标，可建立以下三个典型的优化问题。

其一，全局最小方差组合，寻求风险绝对最小的投资方案：

$$\min_w w^T \Sigma w \quad \text{s.t.} \quad w^T \mathbf{1} = 1$$

其二，给定收益水平下的最小风险组合，在达成收益目标 μ_p 的同时，尽可能控制风险：

$$\min_w w^T \Sigma w \quad \text{s.t.} \quad w^T \mathbf{1} = 1 \quad w^T \mu = \mu_p$$

其三，最大夏普比率组合，引入无风险利率 r_f ，以寻求每单位风险所获得的超额收益最大化的最优风险资产组合：

$$\max_w \frac{w^T \mu - r_f}{\sqrt{w^T \Sigma w}} \quad \text{s.t.} \quad w^T \mathbf{1} = 1$$

以上三个问题共同定义了均值-方差有效前沿，构成了投资组合理论的核心数学模型。

3 算法实现

本项目采用马科维茨均值-方差模型作为资产配置量化框架。该模型旨在解决现代投资决策中的核心命题：“如何在风险约束下实现收益最大化”。为了克服单一求解方法在“精度”与“约束适应性”上的局限性，本项目同步采用了两种截然不同的优化算法进行求解

和对比研究：拉格朗日乘数法（解析解）和蒙特卡洛模拟法（统计近似）。

3.1 算法一：拉格朗日乘数法（解析解）

均值-方差模型是带约束的优化问题，拉格朗日乘数法的核心是：将约束条件融入目标函数，转化为无约束优化问题，通过求偏导为 0 找到极值点。构造拉格朗日函数：

$$L(\omega, \lambda) = f(\omega) + \lambda g(\omega)$$

令：
$$\frac{\partial L}{\partial \omega} = 0, \quad \frac{\partial L}{\partial \lambda} = 0$$

在不考虑非负约束($\omega_i > 0$)的情况下，求解最小方差组合(GMV)和最大夏普比率组合(MSR)，可通过矩阵代数公式直接获得。

3.1.1 最小方差组合

目标：最小化组合方差

$$\begin{aligned}\sigma_p^2 &= \omega^T \Sigma \omega \\ \sum_{i=1}^n \omega_i &= 1\end{aligned}$$

构造拉格朗日函数：

$$L(\omega, \lambda) = \omega^T \Sigma \omega + \lambda (1 - 1^T \omega)$$

对 ω 求偏导并令其为 0，最终最优权重为：

$$\omega_{\text{GMV}} = \frac{\Sigma^{-1} \mathbf{1}}{\mathbf{1}^T \Sigma^{-1} \mathbf{1}}$$

3.1.2 最大夏普比率组合

夏普比率定义为

$$S = \frac{\mu_p - r_f}{\sigma_p}$$

通过拉格朗日法推导（过程类似最小方差组合），最终最优权重为：

$$\omega_{\text{MSR}} = \frac{\Sigma^{-1}(\mu - r_f \mathbf{1})}{\mathbf{1}^T \Sigma^{-1}(\mu - r_f \mathbf{1})}$$

3.2 算法二：蒙特卡洛模拟法（统计近似）

本文采取随机生成 10000 个不同的股票权重组合，确保权重之和为 1 且非负（不允许卖空）。对每个随机组合，计算其年化收益、年化风险（标准差）和夏普比率。从这 10,000 个点中找出具有最高夏普比率的组合作为最优解的近似。这些随机点在图上形成了散点云，直观地展示了所有可能组合的风险-收益分布范围。

权重生成：生成 n 维随机向量 $\omega_{\text{init}} \sim U(0,1)^n$ （ U 为均匀分布），对每组权重归一化，确保 $\sum_{i=1}^n \omega_i = 1$ 且 $\omega_i \geq 0$ 。

指标计算：包括年化收益、年化风险、夏普比率

最优筛选：遍历 10000 组组合，分别筛选出 σ_p 最小的组合(GMV 近似)和 S 最大的组合(MSR 近似)。

3.3 有限前沿对比

图中黑色虚线为拉格朗日算法推导的精确有效前沿，代表所有“给定收益下风险最小”或“给定风险下收益最大”的组合集合；彩色散点为蒙特卡洛模拟的随机组合，其分布范围与有效前沿的重合度体现了模拟的近似精度。红色星号表示拉格朗日求解的最大夏普比率组合，是有效前沿上夏普比率最高的点；蓝色星号表示拉格朗日求解的最小方差组合是有效前沿的左端点（风险最低）；灰色散点表示单资产的风险 - 收益特征，体现了“分散化投资降低风险”的核心价值。

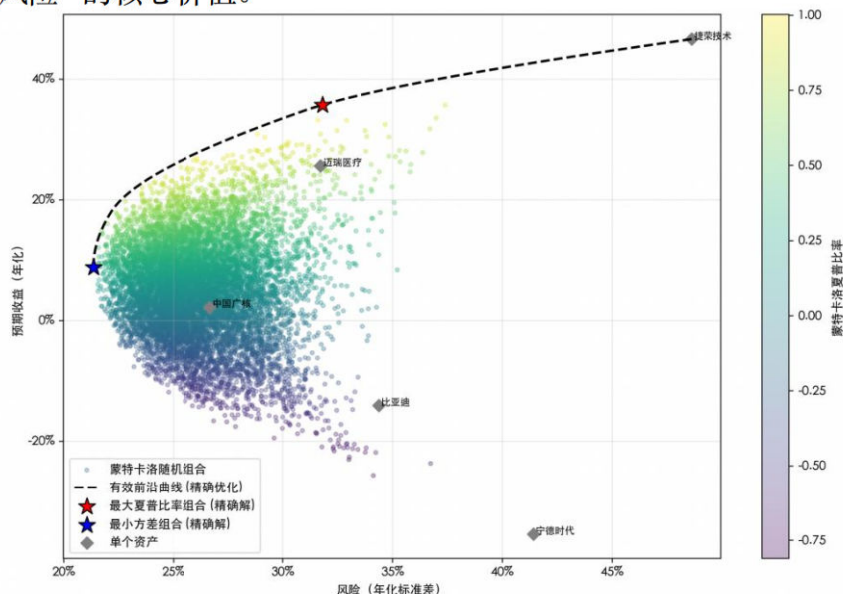


图1 投资组合有效前沿对比：蒙特卡洛 VS 精确优化（拉格朗日）

单个资产指构成整个投资组合的每一支独立的股票本身的风险和收益特征，体现了分散化投资的价值。

4 研究核心结论

拉格朗日解析解提供理论最优解，适用于具备做空与杠杆能力的机构投资者，年化收益高达 546.74%，夏普比率 2.05，但波动剧烈（最大回撤 -58.92%）。而蒙特卡洛模拟更稳健，适合个人投资者，年化收益 29.03%，夏普比率 0.99，最大回撤仅 -25.67%，更具实操性。

4.1 策略建议

这种特性差异直接决定了它们在实际投资中的不同应用场景。对于追求极致收益、具备完善风控体系的机构投资者，拉格朗日解析解支持下的激进型量化策略是更优选择。该策略通过超配高成长个股、适度做空弱势标的构建对冲组合，并辅以 3.0-5.0 倍的动态杠杆管理和日度 VaR 监控，在承担可控风险的前提下争取收益最大化。

而对大多数个人投资者而言，基于蒙特卡洛模拟的稳健型配置策略更具现实意义。该策略通过分散化投资降低单一资产风险，严格执行单票不超过 55% 的仓位控制，

附录 1:蒙特卡洛算法代码实现

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# ===== 关键修改: 设置中文字体 =====
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"] # 支持中文
plt.rcParams["axes.unicode_minus"] = False # 正确显示负号 (避免负号变成方块)

# 步骤1: 读取Excel数据
df = pd.read_excel("合并后的股票数据.xlsx")
df.set_index(keys="日期", inplace=True) # 日期作为行索引
print("原始数据 (前5行): ")
print(df.head())

# 步骤2: 计算核心指标
# 计算日收益率
daily_returns = df.pct_change().dropna()
print("\n年化收益率 (前5行): ")
print(daily_returns.head())

# 计算年化预期收益率 (假设252个交易日)
annual_returns = daily_returns.mean() * 252
print("\n年化预期收益率: ")
print(annual_returns)

# 计算年化协方差矩阵
cov_matrix = daily_returns.cov() * 252
print("\n年化协方差矩阵: ")
print(cov_matrix)

# 步骤3: 生成随机投资组合
risk_free_rate = 0.02 # 无风险利率 (示例: 2%)
num_portfolios = 10000 # 生成10000个组合
num_stocks = len(df.columns) # 股票数量

# 存储结果: [收益, 风险, 夏普比率, 各股票权重]
results = np.zeros((3 + num_stocks, num_portfolios))

for i in range(num_portfolios):
    # 生成随机权重并归一化
    weights = np.random.random(num_stocks) # 随机约束: 权重非负
    weights /= np.sum(weights) # 约束: 权重之和=1

    # 计算组合收益、风险、夏普比率
    port_return = np.sum(weights * annual_returns)
    port_risk = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
    sharpe_ratio = (port_return - risk_free_rate) / port_risk

    # 存储结果
    results[0, i] = port_return
    results[1, i] = port_risk
    results[2, i] = sharpe_ratio
    results[3:3 + num_stocks, i] = weights

# 步骤4: 寻找最优组合 (优化过程)
# 优化目标1: 最大夏普比率组合
max_sharpe_idx = np.argmax(results[2])
max_sharpe_return = results[0, max_sharpe_idx]
max_sharpe_risk = results[1, max_sharpe_idx]
max_sharpe_weights = results[3:3 + num_stocks, max_sharpe_idx]

# 2. 优化目标最小方差组合
min_vol_idx = np.argmin(results[1])
min_vol_return = results[0, min_vol_idx]
min_vol_risk = results[1, min_vol_idx]
min_vol_weights = results[3:3 + num_stocks, min_vol_idx]

# 打印最优组合
print("\n==== 最大夏普比率组合 (风险调整后收益最高) ===")
for stock, weight in zip(df.columns, max_sharpe_weights):
    print(f"{stock}: {weight:.2%}")
print(f"收益: {max_sharpe_return:.2%}, 风险: {max_sharpe_risk:.2%}, 夏普比率: {results[2, max_sharpe_idx]:.2f}")

# 步骤5: 可视化
plt.figure(figsize=(12, 8))

# 绘制所有随机组合
plt.scatter(
    results[1, :], results[0, :],
    c=results[2, :], cmap='viridis',
    marker='o', s=10, alpha=0.7
)
plt.colorbar(label='夏普比率') # 中文标签

# 标记最大夏普比率组合
plt.scatter(
    max_sharpe_risk, max_sharpe_return,
    color='red', s=100, marker='*',
    label='最大夏普比率组合' # 中文标签
)

# 标记最小方差组合
plt.scatter(
    min_vol_risk, min_vol_return,
    color='blue', s=100, marker='*',
    label='最小方差组合' # 中文标签
)

# 标记单个股票的风险-收益
for i, stock in enumerate(df.columns):
    stock_risk = np.sqrt(cov_matrix.iloc[i, i])
    stock_return = annual_returns.iloc[i]
    plt.scatter(stock_risk, stock_return, s=100, label=stock)

# 设置坐标轴格式为百分比
ax = plt.gca() # 获取当前坐标轴对象
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.0%}'))

# 图表标题和标签 (中文)
plt.title('投资组合有效前沿 (均值-方差模型)') # 中文标题
plt.xlabel('风险 (年化标准差)') # 中文x轴标签
plt.ylabel('预期收益 (年化)') # 中文y轴标签
plt.legend() # 显示图例 (含中文)
plt.grid(visible=True, alpha=0.3)
plt.show()
```


附录2 拉格朗日算法实现

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

# 设置中文字体
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False # 正确显示负号

# 步骤1: 读取Excel数据 (与原代码一致)
df = pd.read_excel("合并后的股票数据.xlsx")
df.set_index(keys="日期", inplace=True)
print("原始数据 (前5行): ")
print(df.head())

# 步骤2: 计算核心指标 (与原代码一致)
daily_returns = df.pct_change().dropna()
print("\n日收益率 (前5行): ")
print(daily_returns.head())

annual_returns = daily_returns.mean() * 252
print("\n年化预期收益率: ")
print(annual_returns)

cov_matrix = daily_returns.cov() * 252
print("\n年化协方差矩阵: ")
print(cov_matrix)

# 步骤3: 拉格朗日乘数法求解最优组合
risk_free_rate = 0.02 # 无风险利率
num_stocks = len(df.columns)
ones = np.ones(num_stocks) # 全1向量
inv_cov = np.linalg.inv(cov_matrix) # 协方差矩阵的逆

# 步骤4: 可视化 (突出解析解结果)
plt.figure(figsize=(12, 8))

# 标记单个股票的风险-收益
for i, stock in enumerate(df.columns):
    stock_risk = np.sqrt(cov_matrix.iloc[i, i])
    stock_return = annual_returns.iloc[i]
    plt.scatter(stock_risk, stock_return, s=100, label=stock)

# 标记解析解的最优组合
plt.scatter(
    max_sharpe_risk, max_sharpe_return,
    color='red', s=200, marker='*',
    label='最大夏普比率组合 (解析解)'
)

plt.scatter(
    min_vol_risk, min_vol_return,
    color='blue', s=200, marker='*',
    label='最小方差组合 (解析解)'
)

# 设置坐标轴为百分比格式
ax = plt.gca()
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.0%}'))
ax.yaxis.set_major_formatter(FuncFormatter(lambda x, _: f'{x:.0%}'))

# 图表标签
plt.title('拉格朗日乘数法求解的最优投资组合')
plt.xlabel('风险 (年化标准差)')
plt.ylabel('预期收益 (年化)')
plt.legend()
plt.grid(visible=True, alpha=0.3)
plt.show()

# 1. 最小方差组合解析解 (不考虑收益约束, 仅最小化风险)
min_var_numerator = inv_cov @ ones # 分子:  $I^{n-1} * 1$ 
min_var_denominator = ones @ inv_cov @ ones # 分母:  $1^T * I^{n-1} * 1$ 
min_vol_weights = min_var_numerator / min_var_denominator # 权重归一化

# 计算最小方差组合的收益和风险
min_vol_return = min_vol_weights @ annual_returns
min_vol_risk = np.sqrt(min_vol_weights @ cov_matrix @ min_vol_weights)
min_vol_sharpe = (min_vol_return - risk_free_rate) / min_vol_risk

# 2. 最大夏普比率组合解析解 (考虑风险调整后收益最大化)
excess_returns = annual_returns - risk_free_rate # 超额收益向量
max_sharpe_numerator = inv_cov @ excess_returns # 分子:  $I^{n-1} * (\mu - r_{f})$ 
max_sharpe_denominator = ones @ inv_cov @ excess_returns # 分母:  $1^T * I^{n-1} * (\mu - r_{f})$ 
max_sharpe_weights = max_sharpe_numerator / max_sharpe_denominator # 权重归一化

# 计算最大夏普比率组合的收益和风险
max_sharpe_return = max_sharpe_weights @ annual_returns
max_sharpe_risk = np.sqrt(max_sharpe_weights @ cov_matrix @ max_sharpe_weights)
max_sharpe_ratio = (max_sharpe_return - risk_free_rate) / max_sharpe_risk

# 打印最优组合结果
print("\n=== 拉格朗日乘数法求解结果 ===")

print("\n=== 最小方差组合 (风险最低) ===")
for stock, weight in zip(df.columns, min_vol_weights):
    print(f"{stock}: {weight:.2%}")
print(f"收益: {min_vol_return:.2%}, 风险: {min_vol_risk:.2%}, 夏普比率: {min_vol_sharpe:.2f}")

print("\n=== 最大夏普比率组合 (风险调整后收益最高) ===")
for stock, weight in zip(df.columns, max_sharpe_weights):
    print(f"{stock}: {weight:.2%}")
print(f"收益: {max_sharpe_return:.2%}, 风险: {max_sharpe_risk:.2%}, 夏普比率: {max_sharpe_ratio:.2f}")
```